GUÍA DEL ALUMNO

Saber quién eres no basta. ¿Qué puedes hacer?

TaskFlow con control de acceso: **la sesión en Pinia, roles y permisos, rutas con permiso, interfaz que se adapta y reglas en el servidor.**

Autorización y control de acceso

Vue 3.5 · Firebase 12 · Pinia

Oscar Alvarado

Frontend Tech Lead

Universidad San Martín de Porres

Lima, Perú

ANTES DE EMPEZAR · QUÉ VAS A ENCONTRAR EN ESTA GUÍA

TaskFlow, ahora con control de acceso

Seguimos con **TaskFlow**, la app de las cuatro sesiones del módulo. Ya tiene **estado** (Pinia), **páginas** (Vue Router) y **cuentas y datos reales** (Firebase). Hoy sabe además **qué puede hacer cada persona**. Cada tema es una página corta de teoría y, enseguida, el código para escribirlo.

Los cuatro bloques de hoy

R1
La sesión en Pinia

¿Quién es el dueño de la sesión?estado de tres valores · watch

R2
Roles y permisos

¿Cómo sabemos qué puede hacer cada usuario?useDocument · puede()

R3
Rutas con permiso

¿Cómo evitamos que escriban una URL que no les corresponde? meta.permiso · beforeEach

R4
Interfaz y reglas

¿Y los botones? ¿Y si saltan la interfaz?<Puede> · get() en las reglas

TaskFlow a lo largo del módulo

SESIÓN	QUÉ SE AGREGA A TASKFLOW
1 · Estado	Stores de tareas, tema y sesión, y Tailwind.
2 · Vue Router	Páginas: proyectos, detalle con pestañas, login, guardias, carga diferida y transiciones.
3 · Firebase	Cuentas reales, tareas en Firestore en tiempo real, reglas de seguridad y modo sin conexión.
4 · Acceso (hoy)	Roles y permisos (admin, miembro, lector) en la sesión, las rutas, la interfaz y las reglas del servidor.

Antes de la clase: los roles en tu proyecto de Firebase

1. Usa **el mismo proyecto de Firebase y el mismo TaskFlow de la sesión 3** (con Authentication de correo y Firestore).
2. Crea **tres cuentas** desde «crear cuenta» (por ejemplo `admin@…`, `miembro@…` y `lector@…`).
3. En la consola: *Authentication* → *Usuarios* y copia el **UID** de cada cuenta.
4. En *Firestore* → *Datos* crea la colección `usuarios`: un documento por cuenta, con el **UID como ID** y un campo `rol` (texto): `admin` para la primera y `miembro` para la segunda. La tercera **no necesita documento**: sin rol, la app la trata como `lector`.

5. En *Firestore → Reglas* agrega, dentro de `match /databases/{database}/documents`, este bloque y pulsa **Publicar**: cada quien puede leer su propio rol y nadie puede escribirlo desde la app.

```
match /usuarios/{uid} {
  allow read: if request.auth.uid == uid;
}
```

Cómo seguir la clase

- Parte de **tu TaskFlow de la sesión 3**: todo el código de hoy se escribe sobre él, en clase.
- Cada bloque muestra al final los archivos principales **tal como quedan**, para que compares con los tuyos.
- Node 22.18 o superior y VS Code con **Vue (Official)**.

Quién es el dueño de la sesión

CONCEPTO defineStore

TaskFlow ya tiene cuentas reales, pero cada pantalla pregunta por su cuenta si hay usuario. ¿Quién es el dueño de la sesión?

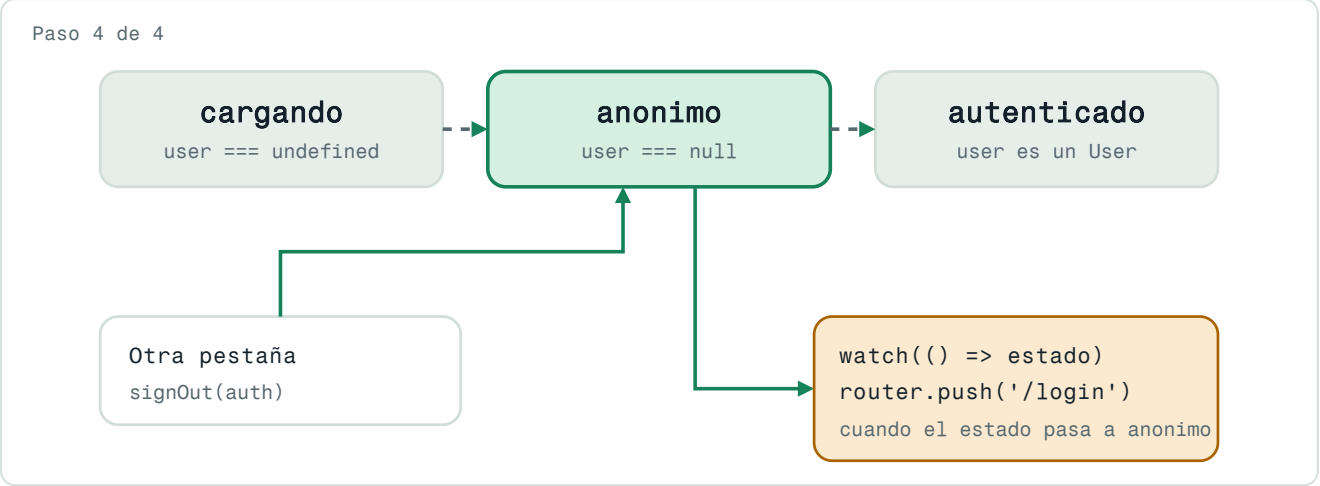
QUÉ ES

Un store de Pinia que guarda **en qué estado está la sesión**: `cargando`, `anonimo` o `autenticado`. Firebase la recuerda en el navegador; el store la expone al resto de la app.

PARA QUÉ SE USA

Que toda la app lea la misma respuesta a «¿quién soy?» y reaccione cuando cambia, también si cierras sesión en otra pestaña.





Cómo funciona, paso a paso

1. Tres estados
2. Firebase la recuerda
3. Un solo dueño
4. Reaccionar al cambio

REGLA

Mientras la sesión **carga** no se sabe si hay usuario: no pintes «Entrar» ni expulses a nadie todavía.

LO QUE AGREGAMOS A TASKFLOW

- estado `cargando · anonimo · autenticado`
- expulsión al cerrar sesión en otra pestaña

Bloque R1 de 4 · La sesión en Pinia

1. **stores/sesion.ts** — añade `type EstadoSesion = 'cargando' | 'anonimo' | 'autenticado'` y un `computed` que lo calcula desde `useCurrentUser()`: `undefined` es cargando, `null` es anónimo. `autenticado` pasa a depender de ese estado.
2. **App.vue** — un `watch` sobre `sesion.estado`: si pasa a `'anonimo'` en una ruta con `requiereAuth`, `router.push` al login.
3. **BarraSuperior.vue** — ⚡ `salir()` ya no navega a mano: lo hace `App.vue`.
4. **Navegador** — abre dos pestañas y pulsa «Salir» en una: la otra va sola al login.

OJO

La guardia del router solo corre **al navegar**. Una sesión que termina sin que navegues (por ejemplo, al cerrar sesión en otra pestaña) la cubre el `watch` sobre el estado.

src/stores/sesion.ts

```
import { computed } from 'vue'
import { defineStore } from 'pinia'
import {
  GoogleAuthProvider,
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  signInWithPopup,
  signOut,
} from 'firebase/auth'
import { useCurrentUser, useFirebaseAuth } from 'vuefire'
import type { Usuario } from '../types'

export type EstadoSesion = 'cargando' | 'anonimo' | 'autenticado'

export const useSesionStore = defineStore('sesion', () => {
  const auth = useFirebaseAuth()!
  const actual = useCurrentUser() // undefined mientras carga · null sin sesión

  // Tres estados en lugar de un booleano: al arrancar aún no sabemos si hay usuario
  const estado = computed<EstadoSesion>(() => {
    if (actual.value === undefined) return 'cargando'
    return actual.value ? 'autenticado' : 'anonimo'
  })

  // Firebase no trae roles: en la sesión 4 vendrán de los claims. Por ahora todos son
   admin.
  const usuario = computed<Usuario | null>(() =>
    actual.value
      ? {
          id: actual.value.uid,
          nombre: actual.value.displayName ?? actual.value.email ?? 'Tú',
```

```
          rol: 'admin',
        }
      : null,
  )
  const autenticado = computed(() => usuario.value !== null)
  const esAdmin = computed(() => usuario.value?.rol === 'admin')

  const registrar = (correo: string, clave: string) =>
    createUserWithEmailAndPassword(auth, correo, clave)
  const entrar = (correo: string, clave: string) => signInWithEmailAndPassword(auth, correo,
   clave)
  const entrarConGoogle = () => signInWithPopup(auth, new GoogleAuthProvider())
  const salir = () => signOut(auth)

  return { estado, usuario, autenticado, esAdmin, registrar, entrar, entrarConGoogle, salir
   }
})
```

src/App.vue

```
<script setup lang="ts">
import { watch } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import BarraSuperior from './components/BarraSuperior.vue'
import { useSesionStore } from './stores/sesion'

const sesion = useSesionStore()
const route = useRoute()
const router = useRouter()

// La sesión puede terminar en cualquier momento (también desde otra pestaña): si pasa en
   una
// pantalla privada, volvemos al login
watch(
  () => sesion.estado,
  (estado) => {
    if (estado === 'anonimo' && route.meta.requiereAuth) router.push({ name: 'login' })
  },
)
</script>

<template>
  <div class="min-h-screen bg-slate-50 text-slate-800 dark:bg-slate-900 dark:text-slate-
   100">
    <BarraSuperior />

    <main class="mx-auto flex max-w-2xl flex-col gap-4 p-6">
      <RouterView />
    </main>
  </div>
</template>
```

Qué puede hacer cada usuario

CONCEPTO useDocument

Ya sabemos quién es cada usuario. ¿Cómo sabemos qué puede hacer: ver, editar o eliminar?

QUÉ ES

Un **rol** es una etiqueta (admin, miembro, lector) y un **permiso** es una acción concreta (`tarea:eliminar`). El rol se guarda en un documento de Firestore (`usuarios/{uid}`) y una tabla lo traduce a permisos.

PARA QUÉ SE USA

Preguntar «¿puede eliminar?» en lugar de «¿es admin?»: si mañana cambian los roles, solo cambia la tabla.

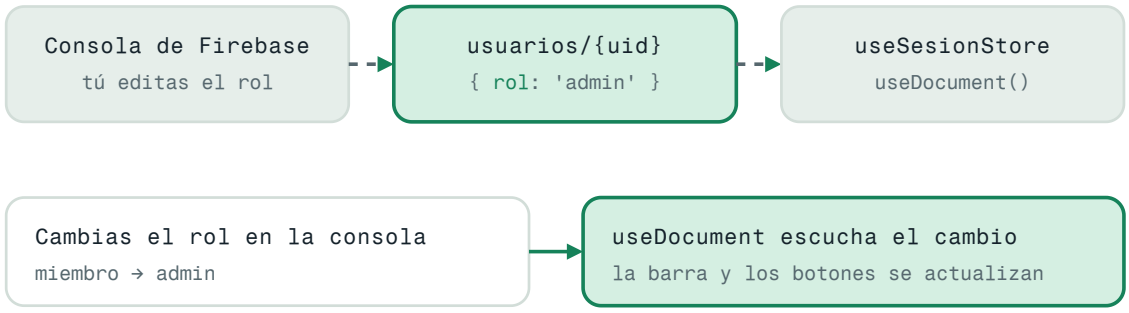
Paso 1 de 4



Sin documento, el mínimo: lector

En producción el rol suele ir como «claim» del token, escrito por un servidor (setCustomUserClaims)

Paso 4 de 4



sin recargar y sin volver a entrar

Cómo funciona, paso a paso

1. El rol vive en la base
2. Tabla rol → permisos
3. puede() en el store
4. Cambia en vivo

REGLA

El cliente **lee** el rol, nunca lo decide: solo se escribe desde la consola de Firebase, y las reglas impiden que la app lo modifique.

LO QUE AGREGAMOS A TASKFLOW

- rol leído de `usuarios/{uid}`
- tabla de permisos tipada
- `puede('tarea:eliminar')`

Bloque R2 de 4 · Roles y permisos

1. **lib/permisos.ts** — tipo `Permiso` y la tabla `PERMISOS: Record<Rol, Permiso[]>`. En `types.ts`, `Rol` suma `'lector'` ⚡.
2. **stores/sesion.ts** — `useDocument` lee `usuarios/{uid}` y `rol` sale de ahí (sin documento, `'lector'`). Añade `puede(permiso)` y `lista()`. `estado` sigue en cargando hasta que llega el rol.
3. **router/index.ts** — ⚡ la guardia espera `await sesion.lista()` en lugar de `getCurrentUser()`.
4. **Navegador y consola de Firebase** — con tu cuenta admin la barra dice «admin». En *Firestore* → *usuarios* cambia `rol` a `miembro`: la barra cambia sola, sin recargar.

OJO

El cliente solo **lee** el rol; quien lo escribe es la consola (y, en R4, las reglas impiden que lo haga la app). En producción el rol suele viajar como *claim* del token, escrito por un servidor; para el cliente es lo mismo.

src/lib/permisos.ts

```ts
import type { Rol } from '../types'

export type Permiso = 'tarea:crear' | 'tarea:avanzar' | 'tarea:eliminar' |
   'proyecto:ajustes'

// Una sola tabla: si mañana cambian los roles, solo se edita aquí
export const PERMISOS: Record<Rol, Permiso[]> = {
  admin: ['tarea:crear', 'tarea:avanzar', 'tarea:eliminar', 'proyecto:ajustes'],
  miembro: ['tarea:crear', 'tarea:avanzar'],
  lector: [],
}
```

src/stores/sesion.ts

```ts
import { computed, watch } from 'vue'
import { defineStore } from 'pinia'
import {
  GoogleAuthProvider,
```

```
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  signInWithPopup,
  signOut,
} from 'firebase/auth'
import { doc } from 'firebase/firestore'
import { useCurrentUser, useDocument, useFirebaseAuth } from 'vuefire'
import { db } from '../services/firebase'
import { PERMISOS, type Permiso } from '../lib/permisos'
import type { Rol, Usuario } from '../types'

export type EstadoSesion = 'cargando' | 'anonimo' | 'autenticado'

export const useSesionStore = defineStore('sesion', () => {
  const auth = useFirebaseAuth()!
  const actual = useCurrentUser() // undefined mientras carga · null sin sesión

  // El rol vive en el documento usuarios/{uid}. Sin documento, el mínimo: lector
  const { data: perfil, error } = useDocument<{ rol: Rol }>(
    () => (actual.value ? doc(db, 'usuarios', actual.value.uid) : null),
    { reset: () => undefined }, // al salir se olvida el rol anterior
  )
  const rol = computed<Rol>(() => perfil.value?.rol ?? 'lector')

  // Tres estados en lugar de un booleano: aún no sabemos quién es, o no hay nadie, o sí
  const estado = computed<EstadoSesion>(() => {
    if (actual.value === undefined) return 'cargando'
    // Falta leer su rol (si la lectura falla, seguimos con el mínimo en vez de quedarnos
   esperando)
    if (actual.value && perfil.value === undefined && !error.value) return 'cargando'
    return actual.value ? 'autenticado' : 'anonimo'
  })

  const usuario = computed<Usuario | null>(() =>
    actual.value && estado.value === 'autenticado'
      ? {
          id: actual.value.uid,
          nombre: actual.value.displayName ?? actual.value.email ?? 'Tú',
          rol: rol.value,
        }
      : null,
  )
  const autenticado = computed(() => usuario.value !== null)
  const esAdmin = computed(() => usuario.value?.rol === 'admin')

  // Preguntamos por la acción, no por el rol: «¿puede eliminar?» y no «¿es admin?»
  const puede = (permiso: Permiso) => PERMISOS[rol.value].includes(permiso)

  // Promesa para las guardias: resuelve cuando ya sabemos quién es y qué rol tiene
  const lista = () =>
    estado.value !== 'cargando'
      ? Promise.resolve()
      : new Promise<void>((ok) => watch(estado, () => ok(), { once: true }))

  const registrar = (correo: string, clave: string) =>
    createUserWithEmailAndPassword(auth, correo, clave)
  const entrar = (correo: string, clave: string) => signInWithEmailAndPassword(auth, correo,
   clave)
  const entrarConGoogle = () => signInWithPopup(auth, new GoogleAuthProvider())
  const salir = () => signOut(auth)

  return {
    estado,
```

```
    rol,
    usuario,
    autenticado,
    puede,
    lista,
    esAdmin,
    registrar,
    entrar,
    entrarConGoogle,
    salir,
  }
})
```

Una guardia, un permiso por ruta

CONCEPTO `meta.permiso`

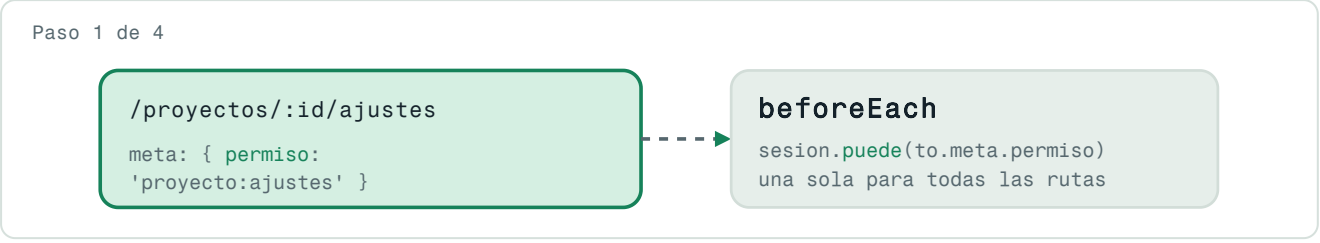
El store ya responde «puede(…)». ¿Cómo evitamos que alguien escriba la URL de una pantalla que no le corresponde?

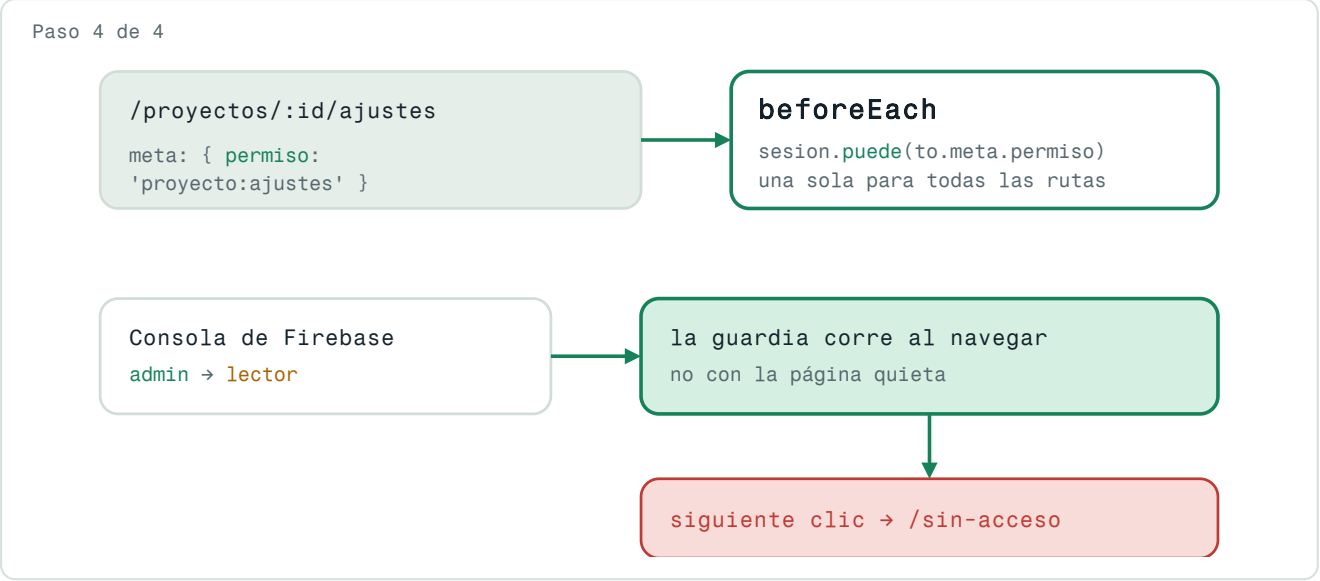
QUÉ ES

Cada ruta declara en `meta` el permiso que exige, y **una sola guardia** lo comprueba con el store. Sin permiso, la navegación va a una página «Sin acceso».

PARA QUÉ SE USA

No repetir un `if` por pantalla y que la regla de acceso quede junto a la ruta.





Cómo funciona, paso a paso

1. La ruta pide un permiso
2. Una guardia para todas
3. Sin sesión o sin permiso
4. El rol cambia con la página abierta

REGLA

Distingue **«no has entrado»** (al login) de **«no tienes permiso»** (a «Sin acceso»).

LO QUE AGREGAMOS A TASKFLOW

- `meta.permiso` tipado
- página «Sin acceso»
- la ruta se revisa en cada navegación

Bloque R3 de 4 · Rutas con permiso

1. **router/index.ts** — en la ruta de ajustes, `meta: { permiso: 'proyecto:ajustes' }` reemplaza a `soloAdmin`. La guardia pregunta `sesion.puede(to.meta.permiso)` y, si no, devuelve `{ name: 'sin-acceso' }`.
2. **router/tipos.ts, SinAcceso.vue** — `RouteMeta` declara `permiso?: Permiso` y se crea la ruta y la página «Sin acceso».
3. **Navegador** — como lector, escribe `/proyectos/1/ajustes`: llegas a «Sin acceso». Como admin se abre.

OJO

Hay dos preguntas distintas: **«¿entraste?»** lleva al login y **«¿tienes permiso?»** lleva a «Sin acceso». Y como `permiso` está tipado, un permiso mal escrito se marca en el editor.

src/router/index.ts

```
import { createRouter, createWebHistory, type RouteLocationNormalized } from 'vue-router'
import { instalarTransiciones } from './transiciones'
import { useSesionStore } from '../stores/sesion'

// Los parámetros de la URL siempre llegan como texto: se convierten aquí, una sola vez
const idDe = (route: RouteLocationNormalized) => Number((route.params as { id: string }).id)

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', name: 'inicio', redirect: { name: 'proyectos' } },
    {
      path: '/proyectos',
      name: 'proyectos',
      component: () => import('../views/ProyectosView.vue'),
      meta: { requiereAuth: true },
    },
    { path: '/login', name: 'login', component: () => import('../views/LoginView.vue') },
    {
      // Ruta padre: pinta el encabezado del proyecto y, dentro, un <RouterView> para el
   hijo
      path: '/proyectos/:id',
      component: () => import('../views/ProyectoView.vue'),
      meta: { requiereAuth: true },
      props: (route) => ({ id: idDe(route) }),
      children: [
        {
          path: '',
          name: 'proyecto-tareas',
          component: () => import('../views/TareasView.vue'),
```

```
          props: (route) => ({ proyectoId: idDe(route) }),
        },
        {
          path: 'ajustes',
          name: 'proyecto-ajustes',
          component: () => import('../views/ProyectoAjustes.vue'),
          meta: { permiso: 'proyecto:ajustes' },
        },
      ],
    },
    {
      path: '/sin-acceso',
      name: 'sin-acceso',
      component: () => import('../views/SinAcceso.vue'),
    },
    // Siempre al final: captura cualquier URL que no coincidió con las anteriores
    {
      path: '/:pathMatch(.*)*',
      name: 'no-encontrada',
      component: () => import('../views/NoEncontrada.vue'),
    },
  ],
})

// Guardia global: corre antes de CADA navegación. Devolver una ruta la redirige; no
   devolver nada la deja pasar.
router.beforeEach(async (to) => {
  // Firebase tarda un instante en saber quién es el usuario y qué rol tiene: la guardia
   espera
  const sesion = useSesionStore()
  await sesion.lista()

  if (to.meta.requiereAuth && !sesion.autenticado) {
    // Guardamos a dónde quería ir para volver después de entrar
    return { name: 'login', query: { redirect: to.fullPath } }
  }
  // Hay sesión, pero ¿tiene permiso? Es otra respuesta: no es el login, es «Sin acceso»
  if (to.meta.permiso && !sesion.puede(to.meta.permiso)) {
    return { name: 'sin-acceso' }
  }
  if (to.name === 'login' && sesion.autenticado) {
    return { name: 'proyectos' }
  }
})

instalarTransiciones(router)
```

src/views/SinAcceso.vue

```
<template>
  <section class="flex flex-col items-center gap-3 py-10 text-center">
    <h2 class="text-2xl font-bold">🔒 Sin acceso</h2>
    <p>Tu rol no puede ver esta pantalla.</p>
    <RouterLink :to="{ name: 'proyectos' }" class="text-sky-600">Volver a los
   proyectos</RouterLink>
  </section>
</template>
```

Que la interfaz se adapte y el servidor lo exija

CONCEPTO `get()`

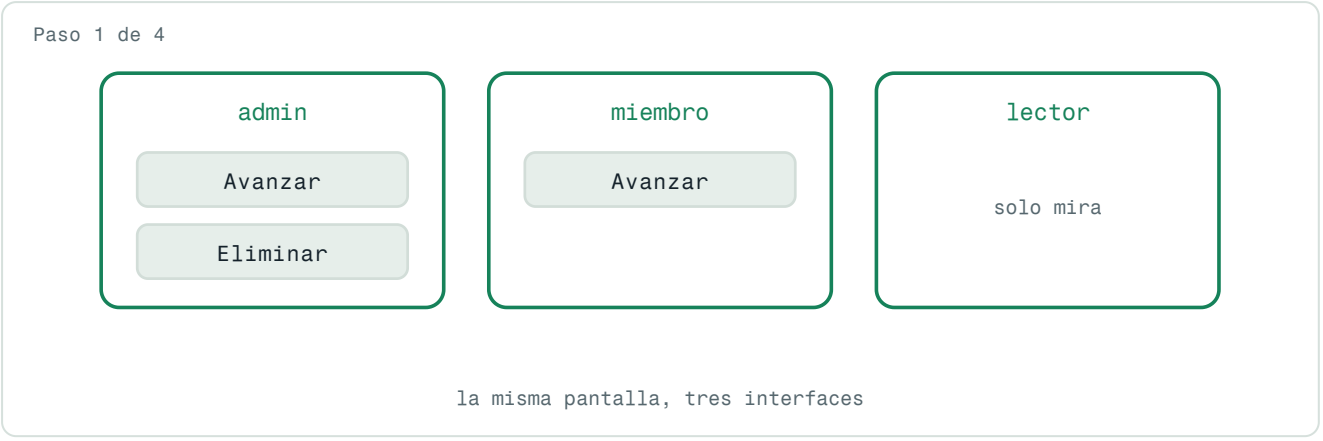
Las rutas ya están protegidas. ¿Y los botones? ¿Y qué pasa si alguien salta la interfaz y llama a Firestore directo?

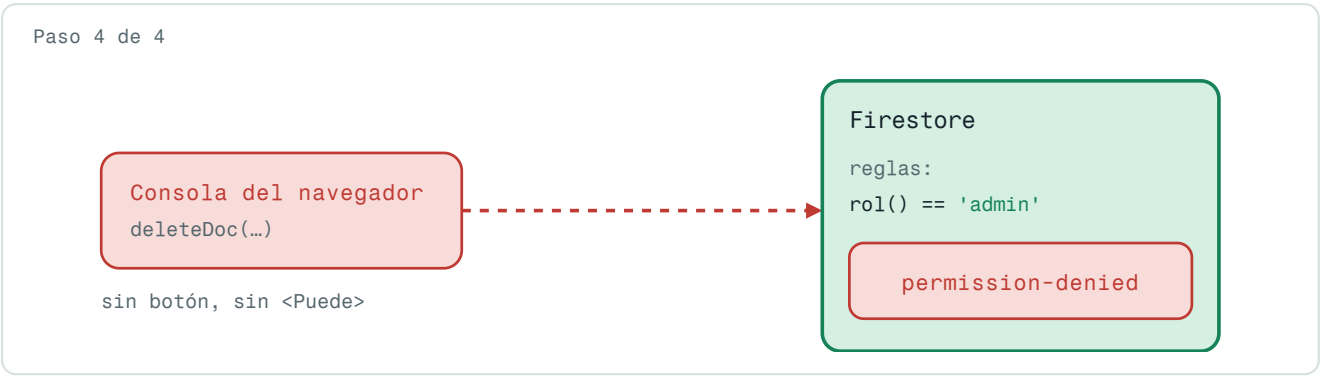
QUÉ ES

Un componente `<Puede>` muestra su contenido solo si el usuario tiene el permiso. Las reglas de Firestore leen el mismo rol con `get()` y deciden en el servidor.

PARA QUÉ SE USA

Una interfaz limpia, sin botones que fallan, y datos que ningún cliente modificado puede saltarse.





Cómo funciona, paso a paso

1. Mostrar solo lo permitido
2. Un componente reutilizable
3. La interfaz se puede saltar
4. El servidor repite la regla

REGLA

Ocultar un botón es **experiencia de usuario, no seguridad**: la decisión de verdad está en las reglas.

LO QUE AGREGAMOS A TASKFLOW

- componente `<Puede>`
- botones según el rol
- reglas que leen el rol

Bloque R4 de 4 · Interfaz y reglas

1. **components/Puede.vue** — un componente sin HTML propio: `<slot v-if="sesion.puede(permiso)" />`, con `permiso` tipado.
2. **TareaCard, TareasView, ProyectoView** — ⚡ «Eliminar», «Avanzar», el formulario y la pestaña «Ajustes» van dentro de `<Puede permiso="…">`.
3. **Consola → Firestore → Reglas** — escribe `rol()` con `get()` y úsalo en `tareas`, luego **Publicar**: crear y editar exigen admin o miembro; eliminar, solo admin.
4. **Navegador** — entra como lector: no ves «Agregar», «Avanzar» ni «Eliminar». Aun llamando a `deleteDoc` desde la consola, Firestore responde `permission-denied`.

OJO

Ocultar un botón es **experiencia de usuario, no seguridad**: quien abra la consola del navegador se salta la interfaz. Lo que protege los datos es la regla del servidor, que lee el mismo rol con `get()` del documento `usuarios/{uid}`.

src/components/Puede.vue

```
<script setup lang="ts">
import type { Permiso } from '../lib/permisos'
import { useSesionStore } from '../stores/sesion'

defineProps<{ permiso: Permiso }>()
const sesion = useSesionStore()
</script>

<template>
  <slot v-if="sesion.puede(permiso)" />
</template>
```

src/firestore.rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // El rol vive en usuarios/{uid}. Sin documento, get() falla y la regla niega: el
   mínimo
    function rol() {
      return
   get(/databases/$(database)/documents/usuarios/$(request.auth.uid)).data.rol;
    }

    // Cada quien lee su propio rol. Nadie lo escribe desde la app: solo desde la
   consola
    match /usuarios/{uid} {
      allow read: if request.auth.uid == uid;
    }

    match /tareas/{tareaId} {
      allow read: if resource.data.uid == request.auth.uid;
      allow create: if request.resource.data.uid == request.auth.uid
                    && rol() in ['admin', 'miembro'];
      allow update: if resource.data.uid == request.auth.uid
                    && request.resource.data.uid == request.auth.uid
                    && rol() in ['admin', 'miembro'];
      allow delete: if resource.data.uid == request.auth.uid && rol() == 'admin';
    }
  }
}
```

CIERRE · PARA SEGUIR

Cuatro preguntas, cuatro respuestas

PREGUNTA	RESPUESTA
¿Quién es el dueño de la sesión?	un store de Pinia con el estado `cargando` · `anonimo` · `autenticado`
¿Cómo sabemos qué puede hacer un usuario?	el rol en `usuarios/{uid}` (`useDocument`) y `puede('tarea:eliminar')`
¿Cómo protegemos las URL?	`meta.permiso` y una sola guardia → «Sin acceso»
¿Y los botones, y los que saltan la interfaz?	`<Puede>` en la interfaz y reglas con `get()` en el servidor

Para practicar

Agrega el permiso `proyecto:crear` solo para admin: ocúltalo en la interfaz con `<Puede>`, protege la ruta con `meta.permiso` y comprueba con un usuario lector que la URL directa lleva a «Sin acceso».

Glosario

TÉRMINO	EN UNA FRASE
Autenticación · autorización	Saber *quién* eres · decidir *qué puedes hacer*.
Estado de la sesión	Cargando, anónimo o autenticado: tres valores, no dos.
Claim	Dato (como el rol) que un servidor puede escribir dentro del token del usuario: la opción habitual en producción.
Rol · permiso	Etiqueta de un grupo de usuarios · acción concreta que se puede permitir.
Cambio en vivo	Si el rol está en un documento, `useDocument` lo escucha y la app se actualiza sola.
Guardia de navegación	Función que corre antes de cada navegación y puede redirigirla.
Reglas de seguridad	Código en el servidor que decide quién lee y escribe cada documento.
Mínimo privilegio	Sin rol asignado, el usuario recibe el rol que menos puede hacer (`lector`).

Para repasar

Claims personalizados (producción)	`firebase.google.com`
Leer otros documentos en las reglas	`firebase.google.com`
Guardias de navegación	`router.vuejs.org`